

Where the attentional bias enters

Model III and Model IV of the hypercolumn / point-set model

2026-08-20 · second pass, rewritten against the code

The answer, first

Two versions of the model differ in exactly one respect: where the top-down attentional bias is applied. They are not new and they are not unnamed. They are Model III (bias on the drive) and Model IV (bias into the pool), built and compared on 2026-07-19, and they are implemented behind a single boolean, `prm.biasInPool`.

Exactly two engines implement both routes: `toy_color.m` (two surfaces) and `pointset/hcps_grid.m` (the 121-point-set grid). Every other file that mentions the flag is a driver that sets it. The locked operating point sets it TRUE — `hcps_op.m` line 148 — so every published HC/PS number, and every figure on the website, is Model IV.

The first pass of this note closed by saying the two were not cleanly separated in our code and that `ps_pointset.py` might be running Model III. That was a misreading. `ps_pointset.py` is a different model — motion only, two V1 hypercolumns, no colour hypercolumn, global normalization — and its FB term is like-to-like MT->V1 stimulus feedback, not the top-down bias.

Common scaffolding

Index a point-set by p , an attribute by a (motion or colour), and a channel within an attribute by θ — eight directions, eight hues.

$u_{\theta}^a(p, t)$ feedforward stimulus drive to a channel

a_{θ} top-down bias: a graded von Mises bump, 0 to `biasAmp`

$S(p, t)$ cooperative pool — ONE per point-set, shared across attributes

$G(p, t) = 1 + \text{CoopL}S(p, t)$ the cooperative gain

Both routes then normalize identically. The drive is raised to the power n BEFORE it is pooled, over channels and then over space:

$$R_{\theta}^a(p) = \frac{R_{\max} [D_{\theta}^a(p)]^n}{\sigma^n + w \sum_{p'} W_n(p, p') \sum_{\theta'} [D_{\theta'}^a(p')]^n}$$

The denominator is FLAT across channels — one per point-set per attribute — because `normKapDir` and `normKapHue` are 0, which makes the featural kernel all-ones. That is a parameter, not a structural property. It is also WITHIN attribute (`featureNorm` true), so motion and colour have separate denominators. Page 6 shows that this last choice is what makes Model IV's headline prediction contingent rather than structural.

The two routes, in the code's own algebra

Verbatim from hcps_grid.m lines 222-240 and toy_color.m lines 67-75, which agree with each other.

Model III — bias on the drive

The bias joins the cooperative term inside a single gain factor on the feedforward drive. The pool integrates the cells' plain responses.

$$D_{\theta}^a = u_{\theta}^a (1 + a_{\theta} + \text{CoopLS})$$

$$\tau_S \dot{S} = -S + \sum_{\theta} R_{\theta}^m + \sum_{\theta} R_{\theta}^c$$

Model IV — bias into the pool

The drive carries no bias at all. The bias re-weights what the pool neuron sees, and the pool returns one scalar gain to every channel of both hypercolumns.

$$D_{\theta}^a = u_{\theta}^a (1 + \text{CoopLS})$$

$$\tau_S \dot{S} = -S + \sum_{\theta} (1 + a_{\theta}) R_{\theta}^m + \sum_{\theta} R_{\theta}^c$$

Four corrections to the first pass

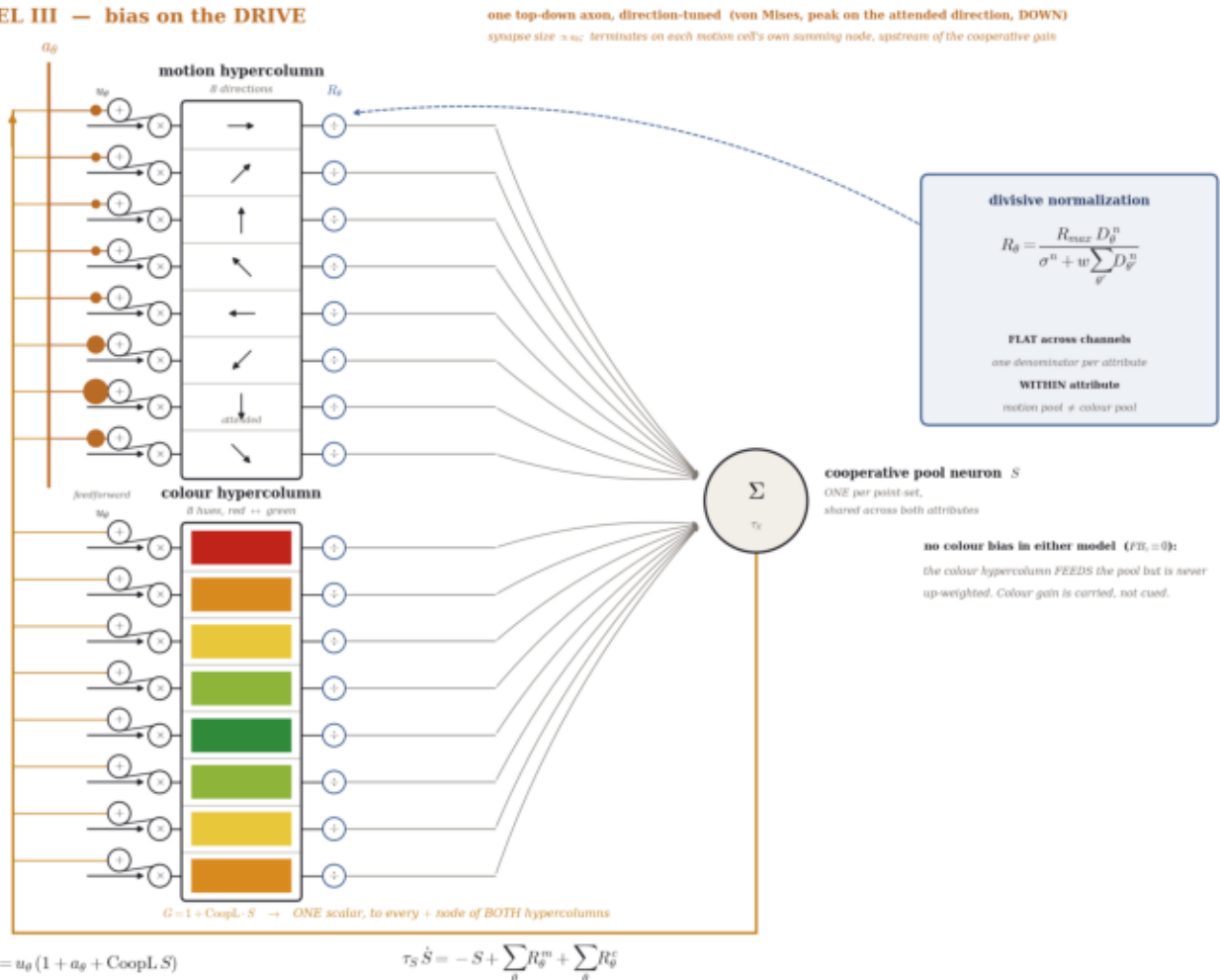
	first pass said	the code does
1	$D \cdot A \cdot G$, multiplicative	$u(1 + a + \text{CoopLS})$ — ADDITIVE inside one gain
2	pool integrates $A \cdot D$, the drive	pool integrates $(1 + a)R$, the RESPONSE
3	a saturating $S()$ on the pool input	pool input is linear; poolAct sits on its OUTPUT
4	A is $1+\beta$ on one channel	a_{θ} is a graded von Mises bump over all eight

Correction 1 matters: the multiplicative form carries a cross-term $a \cdot \text{CoopL} \cdot S$ that the code does not have, so the first pass's clean "factor A" prediction does not follow from what we run. Correction 2 matters for the wiring: weighting R rather than D puts Model IV's synapse downstream of the divisive normalization, not upstream.

Model III as circuitry — bias on the drive

One point-set: a motion hypercolumn and a colour hypercolumn sharing one cooperative pool neuron. Every operation in the equations above is drawn as a node. The tuned top-down axon terminates on each motion cell's own summing node, upstream of the cooperative gain, so the attended channel's own response is raised.

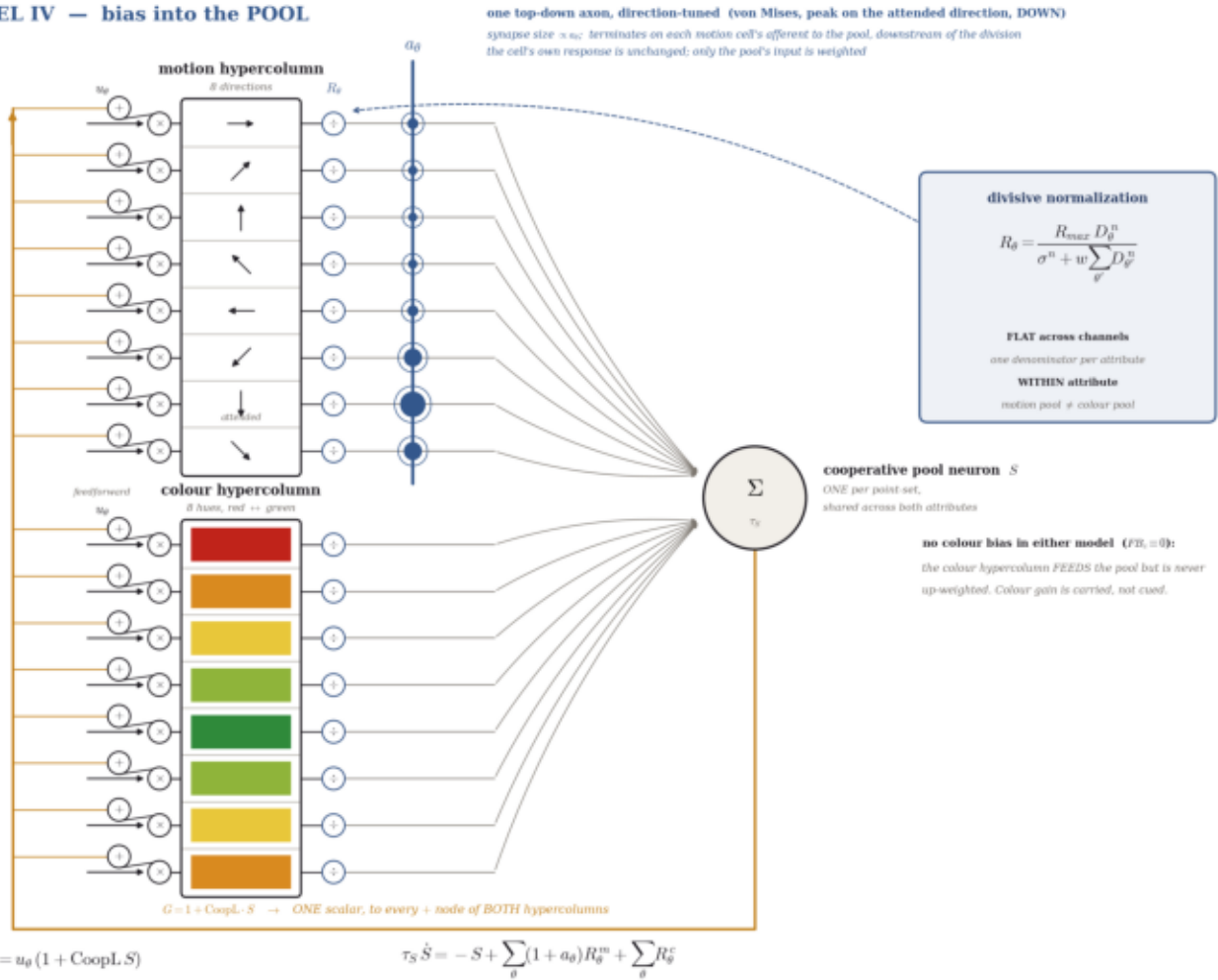
MODEL III — bias on the DRIVE



Model IV as circuitry — bias into the pool

The same drawing, with one thing moved. The identical tuned axon now terminates on each motion cell's afferent to the pool, downstream of the division. The V1 cells themselves are unbiased; only what the pool neuron sees is weighted. Compared with page 3, the hypercolumns, the pool, the normalizer and the afferents are pixel-identical.

MODEL IV — bias into the POOL



$$D_\theta = u_\theta (1 + \text{CoopL} \cdot S)$$

REQUIRES the same tuned projection modulating each V1 $\rightarrow$ pool afferent presynaptically. The V1 cells are unbiased; only the pool's input is weighted.

What each predicts, and what it measures

Under Model IV every channel of a point-set is multiplied by the same scalar G , so for any channel the cued/uncued ratio is $(G_c/G_u)^n$ divided by the ratio of the denominators, and the per-channel drive cancels. If the motion and colour denominators move together, the attention index is then IDENTICAL for motion and for colour. Under Model III the attended channel additionally carries a θ , so colour can only gain second-hand, through the pool.

Measured with pointset/hcps_bias_locus_check.m, 8 seeds, reading primary (motion DOWN), colour (V4 GREEN) and translation (motion RIGHT). Model III and Model IV are shown at MATCHED primary AI in the toy, where biasAmp can be bisected; on the grid both run at the locked biasAmp 16, so compare the colour/primary RATIO rather than the raw colour number.

toy_color.m — 2 surfaces, one shared normalizer (featureNorm false)

route	biasAmp	primary	colour	translation
Model III on drive	1.00	+0.6081	+0.1022	+0.2321
Model IV in pool	9.63	+0.6081	+0.6081	+0.4886

Model IV's primary and colour agree to $1.1e-16$ — machine epsilon. Model III's colour is 16.8% of its primary: a real transfer, but a small one.

hcps_grid.m — 121 point-sets at the LOCKED operating point (featureNorm true)

route	primary	colour	translation	colour/primary
Model III on drive	+0.9875	+0.0389	+0.8141	3.9%
Model IV in pool	+0.1996	+0.1996	+0.2058	100.0%

So the separation is not subtle. Model III sends 4% of its motion enhancement to colour; Model IV sends all of it. logs/PARAMETERS.md line 61 says Model III gives "motion enhancement with no colour transfer" — that is right to within noise on the grid with a shared normalizer (colour AI -0.0005), an understatement at the locked point (3.9%), and wrong at the toy scale (16.8%). The qualitative claim holds everywhere; the magnitude is regime-dependent and should be quoted with its regime.

One regime is unusable and is reported here so it is not tried again: with normW = 0 the toy has no denominator, runs away (max response $1e115$) and saturates every index to ± 1 . Pure facilitation needs the normalizer to stay bounded — the Model I to Model II lesson.

The equality is contingent, not structural

The first pass claimed that no free parameter can move Model IV off primary = colour, and that this is what makes the comparison worth running. The website's viewer caption makes the same claim, saying the two indices "land on top of each other to three decimals, and that equality IS the transfer". At the locked operating point that claim is false as stated.

Model IV's equality is exact only when the motion and colour denominators move together. With featureNorm FALSE they are literally the same variable (hcps_grid.m lines 251-252), so the equality is structural and no parameter touches it. With featureNorm TRUE — the locked default — they are separate pools, and the equality survives only because the two drive profiles happen to have the same shape: ffNorm rescales both to the same total, and kappa equals kappaHue equals 2, so their sums of powers coincide.

Tested by detuning the hue width alone, everything else held at the locked point (pointset/hcps_bias_locus_ablate.m):

kappaHue	primary	colour	primary - colour	profile mismatch
2.00 (= kappa)	+0.199648	+0.199648	2.78e-17	8.6e-16
2.50	+0.199570	+0.198723	8.47e-04	2.1e-01
4.00	+0.199214	+0.196691	2.52e-03	8.9e-01
8.00	+0.198586	+0.194135	4.45e-03	2.6e+00

The proposed cause and the effect move together: the equality leaves machine epsilon exactly when the drive profiles stop matching, and a 25% detune costs four orders of magnitude. So kappaHue IS a free parameter that moves Model IV off equality. The prediction is still sharp and still worth testing — 100% transfer against 4% is not a close call — but it should be stated as resting on matched tuning widths, and the viewer caption should be corrected.

Where each route is implemented

file	what it is
toy_color.m 67-75	2 surfaces; origin of the III / IV distinction
hcps_grid.m 222-240	the 121-point-set grid engine; what the site runs
hcps_grid_adapt.m	same two-site switch (fbSite) for MT->V1 feedback
ps_pointset.py	a DIFFERENT model: motion only, no colour HC

Two drivers already sweep both routes — hcps_bias_route_atop.m and hcps_grid_biasout.m — but both run at stale operating points and neither has a stored result anywhere in the repo. Do not cite them.

Still open: ps_pointset.py's docstring claims an object-based bias on hypercolumn B, while its code applies a feature-based bias to both hypercolumns. And lib/hcpsDefaults.ts on the website still exports ffFloor 1e-06 and hash d0ea82, which hcps_op.m superseded on 2026-07-28.